

MeitY Sovereign AI Infrastructure: Comprehensive Technical Architecture & Component Specification

Document Ref: MEITY-TECH-ARCH-SPEC-V2.0	Date: July 31, 2026
Author: Supratim Dhara (Systems Architect, FlockML)	Engine Specifications: BitNet 1.58-Bit WGSL / WASM SIMD / FedAvg-M

1. COMPREHENSIVE END-TO-END SYSTEM TOPOLOGY

FlockML is an open-source, decentralized machine learning protocol designed to run full training and inference loops locally inside heterogeneous edge devices (desktop PCs, smartphones, laptops, and edge servers) without sending raw user data or requiring dedicated NVIDIA CUDA data centers.



2. DETAILED BREAKDOWN OF SYSTEM COMPONENTS

2.1 Component A: WebGPU WGSL Shader Engine (BitNet 1.58-Bit)

For devices equipped with integrated or discrete GPUs (Apple M-Series, Intel Iris Xe, Qualcomm Adreno, ARM Mali, NVIDIA RTX), FlockML executes matrix multiplications directly on GPU hardware via WebGPU Shading Language (WGSL).

By quantizing model weights W into ternary precision $\{-1, 0, +1\}$ using BitNet 1.58-bit architecture, floating-point multiplications are completely replaced by ternary additions and subtractions:

$$Y_i = \sum_{j:W_{ij}=+1} X_j - \sum_{j:W_{ij}=-1} X_j$$

Exhaustive WGSL Compute Shader Implementation:

```
// FlockML WebGPU WGSL Shader Kernel (Ternary Addition Accumulator)
@group(0) @binding(0) var<storage, read> input_x : array<f32>;
@group(0) @binding(1) var<storage, read> weights_w : array<i32>; // Packed ternary values {-1, 0, 1}
@group(0) @binding(2) var<storage, read_write> output_y : array<f32>;

var<workgroup> tile_x : array<f32, 64>;

@compute @workgroup_size(64)
fn main(@builtin(global_invocation_id) global_id : vec3<u32>, @builtin(local_invocation_id) local_id : vec3<u32>) {
    let row = global_id.x;
    var sum : f32 = 0.0;

    // Process input vector in 64-element shared workgroup tiles
    for (var tile : u32 = 0u; tile < 16u; tile = tile + 1u) {
        let col = tile * 64u + local_id.x;
        tile_x[local_id.x] = input_x[col];
        workgroupBarrier(); // Synchronize shared workgroup tile memory

        for (var k : u32 = 0u; k < 64u; k = k + 1u) {
            let weight_val = weights_w[row * 1024u + (tile * 64u + k)];
            if (weight_val == 1) {
                sum += tile_x[k];
            } else if (weight_val == -1) {
                sum -= tile_x[k];
            }
        }
        workgroupBarrier();
    }
    output_y[row] = sum;
}
```

2.2 Component B: WebAssembly (WASM) Multithreaded SIMD Fallback Engine

For legacy desktops or low-cost smartphones without WebGPU support, FlockML automatically routes computational pipelines to a multithreaded WebAssembly engine utilizing 128-bit SIMD vector instructions (`v128`).

- **Multithreading Architecture:** Uses Web Workers communicating over a unified `SharedArrayBuffer` using `Atomics.wait()` and `Atomics.notify()` primitives to prevent thread contention.
- **Vector Acceleration:** Executes 4 parallel 32-bit floating-point operations per CPU instruction cycle.

2.3 Component C: Origin Private File System (OPFS) Local Storage

To bypass browser JavaScript memory heap limits (typically 2GB to 4GB) and prevent Out-Of-Memory (OOM) browser crashes when loading 3B-parameter models, FlockML streams weight binaries directly to/from the device's SSD via the browser's **Origin Private File System (OPFS)**.

- **Direct-to-Disk Synchronous Handle:** Uses `FileSystemSyncAccessHandle` inside dedicated Web Workers to achieve zero-latency byte offsets (`read()` and `write()`).
- **Zero-Second Outage Recovery:** If the network connection disconnects, local inference continues uninterrupted using the model binary cached inside OPFS.

3. CRYPTOGRAPHIC PRIVACY & QUANTIZATION FIREWALL

3.1 Component D: Laplacian Differential Privacy ($\epsilon = 0.8$) Noise Engine

To guarantee that local user inputs, sensitive documents, or search queries can never be reconstructed from network updates, each client node executes $\$L_1\$$ gradient clipping and injects zero-mean Laplacian noise prior to transmission:

$$\Delta \bar{W} = \frac{\Delta W}{\max\left(1, \frac{\|\Delta W\|_1}{C}\right)}, \quad \tilde{\Delta W} = \Delta \bar{W} + \text{Lap}\left(0, \frac{C}{\epsilon}\right)$$

Parameters: Clipping Threshold $C = 1.0$, Privacy Loss Budget $\epsilon = 0.8$. This provides provable, non-reversible differential privacy guarantees preventing gradient inversion attacks.

3.2 Component E: Int8 Symmetric Quantization Engine

Floating-point gradient updates (Float32) are mapped to 8-bit signed integer space (Int8) using a symmetric scaling factor S :

$$S = \frac{127}{\max(|W_{\min}|, |W_{\max}|)}, \quad Q_{\text{int8}} = \text{round}(\tilde{\Delta W} \times S)$$

This reduces payload transmission size from 4 bytes per parameter to 1 byte per parameter, saving **80% network bandwidth** over inter-node WebSocket connections.

4. SOVEREIGN AGGREGATOR & FEDERATED AVERAGING WITH MOMENTUM (FEDAVG-M)

4.1 Component F: FedAvg-M Global Model Update Engine

The Sovereign Coordinator server receives quantized updates Q_k from K participating edge nodes, dequantizes them ($\hat{\Delta W}_k = Q_k / S_k$), and computes the momentum-accelerated global weight update:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \sum_{k=1}^K \frac{n_k}{N} \Delta \hat{W}_k, \quad W_{t+1} = W_t - \eta \mathbf{v}_{t+1}$$

Where $\beta = 0.9$ is the server momentum hyperparameter, $\eta = 0.01$ is the global learning rate, and n_k/N represents sample weighting.

4.2 Component G: Byzantine Fault Tolerant (BFT) Gradient Pruning

To prevent malicious nodes from poisoning the global model, the server measures the cosine similarity between each incoming node gradient $\Delta \hat{W}_k$ and the median gradient vector $\mathbf{m}_t$:

$$\text{CosSim}(\Delta \hat{W}_k, \mathbf{m}_t) = \frac{\Delta \hat{W}_k \cdot \mathbf{m}_t}{\|\Delta \hat{W}_k\|_2 \|\mathbf{m}_t\|_2}$$

If $\text{CosSim} < 0.2$, the update is flagged as an outlier and immediately discarded before aggregation.

5. COMPLETE WEBSOCKET PROTOCOL MESSAGE SCHEMAS

```
// 1. Client Node Registration Payload (WebSocket Event: 'node:register')
{
  "protocol": "flockml/v2.0",
  "nodeId": "node-nic-delhi-0492",
  "timestamp": 1785493000912,
  "hardwareCapabilities": {
    "executionBackend": "WebGPU_WGSL",
    "gpuVendor": "Apple M2 GPU",
    "simdSupported": true,
    "vramAvailableMB": 8192,
    "opfsStorageAvailable": true
  }
}

// 2. Encrypted Gradient Transmission Payload (WebSocket Event: 'gradient:upload')
{
  "protocol": "flockml/v2.0",
  "nodeId": "node-nic-delhi-0492",
  "roundId": 14,
  "sampleCount": 128,
  "privacyMetadata": {
    "epsilon": 0.8,
    "clippingThreshold": 1.0,
    "quantizationScaleFactor": 142.31
  },
  "quantizedGradientsInt8": [12, -4, 0, 89, -127, 45, 0, -12, 114, -8]
}

// 3. Server Global Weight Broadcast Payload (WebSocket Event: 'weights:broadcast')
{
  "protocol": "flockml/v2.0",
  "globalRound": 15,
  "activeNodeCount": 42,
  "globalLoss": 0.00412,
  "globalWeightsInt8Url": "https://meity-pilot.gov.in/api/weights/round-15.bin"
}
```

6. EXHAUSTIVE SECURITY CONTROL MATRIX

THREAT VECTOR	ATTACK MECHANISM	FLOCKML TECHNICAL CONTROL	VERIFICATION TEST
Gradient Inversion	Reconstructing user text inputs from model gradients	Laplacian Differential Privacy ($\epsilon = 0.8$) adds non-reversible mathematical noise	Wireshark packet capture analysis
Data Poisoning	Malicious node submitting bad weights to ruin global model	Byzantine Fault Tolerant (BFT) Cosine Distance gradient pruning	Outlier node injection simulation
Man-in-the-Middle	Intercepting gradient updates in transit over network	TLS 1.3 encrypted WebSockets (<code>wss://</code>) with strict certificate pinning	Network packet tampering test
Heap Memory OOM	3B parameter weights crashing browser memory	OPFS direct-to-disk synchronous file handle streaming	60-minute stress test on 4GB RAM devices

Exhaustive Architecture Specification Certified By:

Supratim Dhara
Systems Architect & Founder, FlockML
Email: supratimdhara0@gmail.com | Portfolio: <https://supratimdev.qzz.io>